

Fast Collision Detection Method with Octree-Based Parallel Processing in Unity3D [†]

Kunthroza Hor ¹, Taeheon Kim ¹ and Min Hong ^{2,*}

¹ Department of Software Convergence, Soonchunhyang University, Asan-si 31538, Republic of Korea; zoroberto16@gmail.com (K.H.); taeheon1213@gmail.com (T.K.)

² Department of Computer Software Engineering, Soonchunhyang University, Asan-si 31538, Republic of Korea

* Correspondence: mhong@sch.ac.kr

[†] Presented at the 2024 IEEE 7th International Conference on Knowledge Innovation and Invention, Nagoya, Japan, 16–18 August 2024.

Abstract: Performing accurate and precise collision detection is a key to real-time applications in computer graphics, games, physical-based simulation, virtual reality, augmented reality, and research and development. Researchers have developed numerous methods to minimize computation time and enhance the accuracy of collision detection for pair-object collisions. Although the performance of the central processing unit (CPU) has significantly improved in recent years, it is still insufficient for many applications. In this study, we have developed an improved algorithm for geometric bounding volume hierarchy (BVH) in 3D spatial subdivisions using an Octree-based axis-aligned bounding box (AABB) structure. The AABB structure is used for collision detection and its computation by the central processing unit and graphic processing unit (GPU), which is implemented on the compute shader in Unity3D. AABB was defined as the maximum and minimum hexahedron within an object that is parallel to the coordinate axis. While GPU computing is essential for enhancing the object's performance. The proposed algorithm approaches Octree AABB-based GPU parallel processing to reduce the calculation or process of simulation for real-time collision detection and handles multiple computations. In the CPU environment, the algorithm spent 2.9 fps when simulating up to 20 objects of the Torus Model that contains 2.3 K vertices and 4.6 K triangles. In the GPU environment, it spent 635.62 fps with 20 objects, and the maximum number increased to 180 objects in real-time.

Keywords: collision detection; octree; GPU parallel processing; compute shader; unity3D



Academic Editors: Teen-Hang Meen, Chun-Yen Chang and Cheng-Fu Yang

Published: 13 March 2025

Citation: Hor, K.; Kim, T.; Hong, M. Fast Collision Detection Method with Octree-Based Parallel Processing in Unity3D. *Eng. Proc.* **2025**, *89*, 37. <https://doi.org/10.3390/engproc2025089037>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

As one of the most effective techniques for real-time applications, collision detection has been widely used, especially in computer graphics, games, computer animation, virtual reality, augmented reality, and 3D graphics [1]. To develop and detect a well-geometry structured 3D and an accurate real-time simulation, collision detection plays a critical role in intersecting and reaching objects adaptably and effortlessly. Collision detection is utilized to determine objects intersecting with each other in a simulated environment. The Octree axis-aligned bounding box (AABB) [2] represents a geometric bounding volume hierarchy (BVH) in 3D spatial subdivision for the simulation of the rigid objects that are characterized by a pair of objects intersection between the minimum point and maximum point [3]. If the minimum value of AABB exceeds the highest maximum value of the other along any axis, there is no collision along that particular axis. Conversely, if the maximum value of one AABB is less than the minimum value of the other along any axis, they cannot be colliding

along that axis, as collision-free detection occurs at any pair between two objects. If the conditions do not meet the criteria for each of the three axes, the AABB intersections are considered to have collided.

Spatial subdivision has been widely utilized to optimize collision detection by reducing the number of collisions. The main approach is to divide the space into multiple cube regions and subsequently perform collision tests for primitives with the same cell using the Octree structure. Octree is a data structure that subdivides the space into eight equally sized cells or cube boundaries of the three-level hierarchical grid in a model space [4]. To develop and commercialize parallel processing, multicore processors are used as the graphic processing unit (GPU) for the computation of a massively parallel algorithm that allows for faster structure and quickly determines potential interactions between objects with each other at runtime [5]. GPU parallel processing can greatly enhance the processing speed for tasks that can be subdivided into smaller sub-tasks and computed on the GPU kernel program in the compute shader script for rendering.

2. Related Work

There are several studies on collision detection for rigid-body objects. Different algorithms are used to improve the efficiency of collision detection for rigid-body object simulation. Collision detection has been studied extensively in computer graphics, games, computer animation, and 3D graphics. Collision detection is the process of detecting when two or more objects and shapes are about to collide or have already collided. Aissa et al. [6] addressed the Octree-optimized microstructure of a discontinuous fiber for virtual sample generation to carry out precise prediction and simulation. The data of the tree structure are constructed recursively within a computational domain. The tree is constructed in refinement steps, dividing the computational domain into two to generate subdomains (children). This division process generates eight subdomains named Octree, with element allocation to the children managed using AABB. Serpa et al. [7] provided a complete testing technique that comprises several implementations in CPU and GPU environments. Related algorithms are categorized based on spatial and sorting methods. Spatial methods partition the problem using spatial data structures and employing a divide-and-conquer strategy. The data structure and implementation of the spatial structure are known as Grids, bounding-volume hierarchy (BHV), KD-Trees, Octrees, and BSP-Trees. The majority of the techniques reduce the search space by pruning collision pairs of object detections. The concept of GPUs is designed to execute large amounts of mathematical operations in parallel to efficiently operate the input data in a process called a graphics pipeline by utilizing scripts known as shaders [8]. Then, input data are employed to programmatically manipulate and expose it to the parallel, heavily multithreaded GPU hardware. GPU hardware and driver support are developed by introducing a new type of shader called the compute shader for general-purpose computations.

3. Methodology

The purpose of the implementation is to improve and represent collision detection between objects for geometric BHV in 3D spatial subdivision as a simple geometric primitive and vertices. The minimum and maximum bounding boxes are determined to include them in parallel hierarchical computation on the GPU. The algorithm comprises three methods.

3.1. Octree Structure

There are several spatial data structures to organize 2D and 3D data in computer graphics. Among these data structures, Octree is used widely and appears in many extensive application ranges, especially when positions must be accessed and manipulated.

Octree is a structure of increasingly dynamic and adaptable computations to decrease time complexity. Octree construction divides a three-dimensional space adaptively and the space of the children to the distribution of geometric primitives. It reduces the number of empty cells to save storage costs. With the advancement of programmable graphics hardware, Octrees can be constructed and traversed using GPU computation [9]. Octrees leverages the parallel computing feature of GPUs to enhance rendering and geometry processing. Its composition represents the full computational region of an object at the zero level of the octree hierarchy. Then, this region is subdivided into eight sub-regions of the children that present nodes in the first level. The eight sub-regions are defined by eight sub-region children that present nodes in the second level. The three levels of octree provide storage cost using reduced memory since the space partials without collision occurrence are not kept overlapped.

The Octree data structure hierarchically represent volume spaces. The space of the Octree needs to be traversed from the top to bottom to insert the object into the corresponding space nodes. Octants start from the root node in the zero level to the current eight nodes in the first level to the eight nodes in the second level of the spatial hierarchy. The root of the current nodes is defined by ordering the index number to fill the node regions inside the maximum of the Octree size as shown in Figure 1.

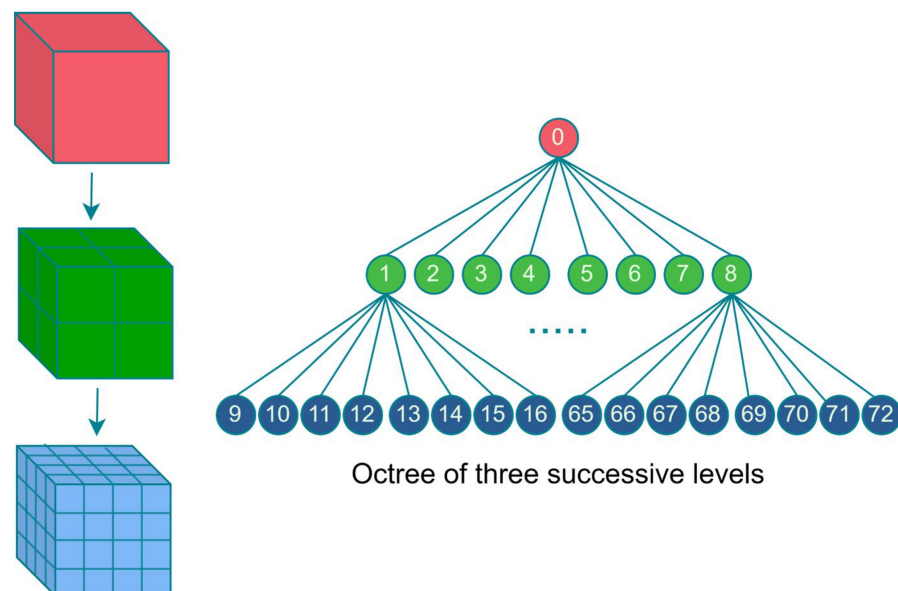


Figure 1. Structure of three successive levels in Octree.

3.2. AABB

AABB represents the basic structure. AABB is the most popular bounding box that has the maximum and minimum values of the vertex of the tetrahedron mesh on the x, y, and z coordinates. Its most outstanding feature is its ability to conduct rapid intersection, achieved by comparing coordinate values only. The method involves iterating through all vertices within the object's bounding box and then examining the projection range to detect overlaps between AABB intersections. This is achieved by comparing the two AABB bounding boxes along the three axes. The AABB is described in the x, y, and z directions to calculate the maximum and minimum along each axis by conducting an AABB intersection test with six comparison operations. The utilization of AABB and octree algorithms reduces test times and pair comparisons, thereby increasing overall efficiency at each level of the Octree structure. Each level has to be computed using six operations to find a collision index pair of objects in each level that are simultaneously intersecting and eliminated from further collision tests as presented in Figure 2.

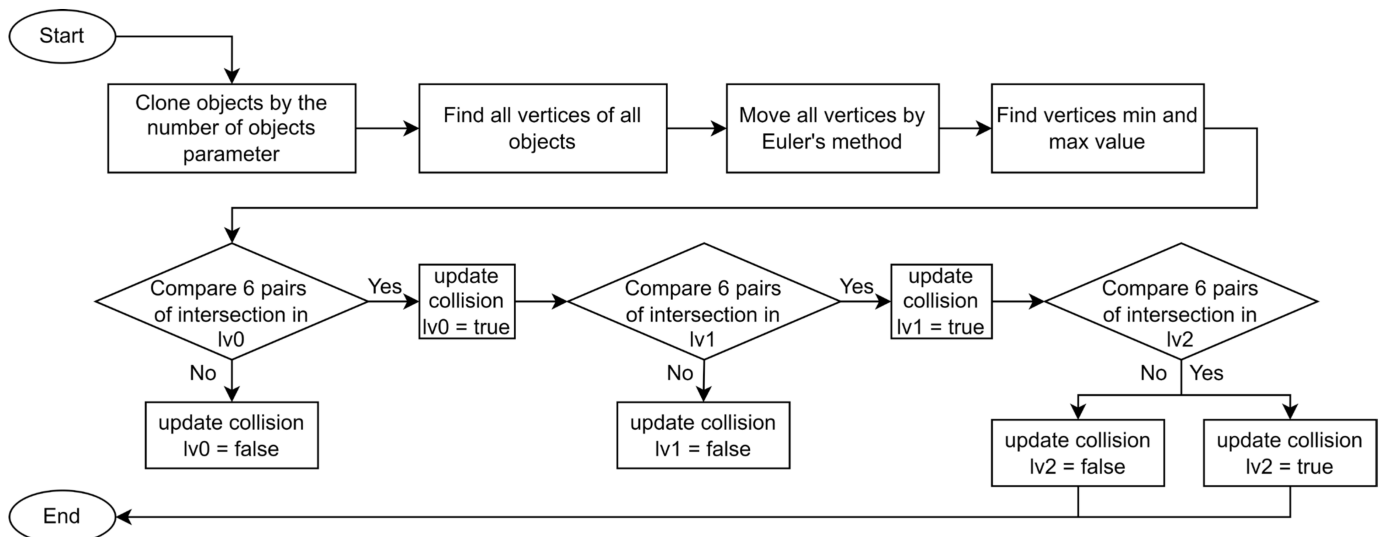


Figure 2. Six times of intersection comparison of minimum and maximum values.

After implementation, all combined vertices of all objects are moved by applying the explicit Euler physical-based method. The object and floor maximum and minimum bounding boxes are determined if there is an overlap between the AABB intersection. After collision detection occurs, the reverse velocity of particles colliding with the floor in the object is updated to make an object movement animation as shown in Figure 3.

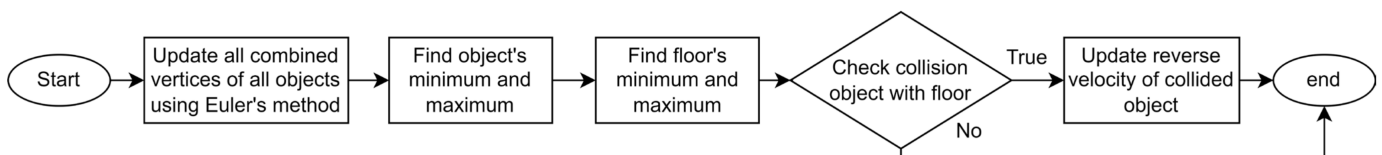


Figure 3. Checking collision detection of bounding box of objects with floor.

3.3. GPU Parallel Processing

Parallel hierarchical computation on the GPU speeds up performance, detection, and mathematical operations due to the nature of hardware platforms. The GPU-based algorithm utilizes parallelism to accelerate collision detection processes, such as general-purpose computing and simulation, by parallelizing the task massively and parallelly in hundreds of threads [10]. The Unity3D game engine is used widely in game development to render pipelines and conduct physical simulations using high-level shader language (HLSL). HLSL is essential for the GPU implementation and design of the programmable shader script operating on virtual works groups comprised of small execution units (cores). The C# programming language is used for scripting to perform operations sequentially. Its efficiency can be enhanced through multi-threaded computing. Shaders are programs tasked with the rendering of graphical data. Within Unity3D, the compute shader expands upon Microsoft Direct3D 11's technology to develop cross-platform applications. The Unity3D engine allows for the creation of a compute shader using a regular script. When invoking or running the application, it calls the dispatch method, which then activates the kernel program for execution within the C# script. The compute shader function is a kernel program specified by the number of workgroup blocks, with each workgroup block being generated by multiple threads. Moreover, the compute shader function as a kernel program is implemented by the number of threads with three parameters on the x, y, and z dimensions of the 3D vectors. The three parameters are represented as one-dimensional, two-dimensional, or three-dimensional computational tasks in parallel. The total number

of threads is the product of the three parameters of the dispatch function. Therefore, the execution of all threads occurs in a non-sequential order, providing an advantage in performance, especially when handling extremely large data as demonstrated in Figure 4.

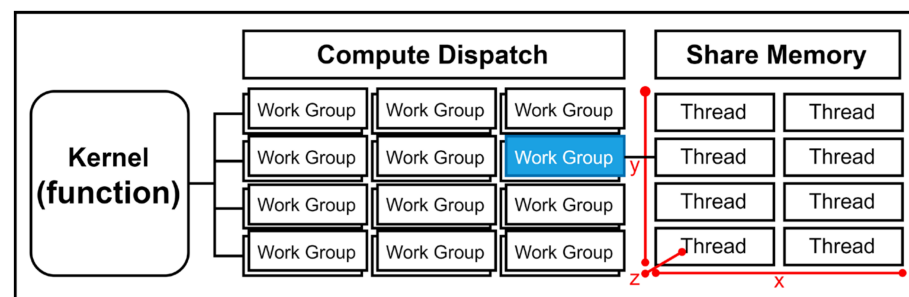


Figure 4. Compute shader elements of kernel in Unity.

4. Result and Discussion

The experiments were conducted using a Windows 10 Pro 64-bit system with an Intel Core™ i7-7700 Processor with 3.60 GHz, 32 GB of RAM, and an NVIDIA GeForce RTX 3070Ti with 24 GB of V-RAM. C#, HLSL shader model 5.0 (Windows Graphics API), and Unity engine version 2020.3.8f1 were utilized to configure the GPU pipeline. CPU-based computations have limitations compared to GPU-based performance, as they are 26 times lower than the GPU, as provided in Table 1. The simulation is calculated 3 times for each object in the same condition, limited by 400 maximum frames in CPU, and limited by 2000 frames in GPU. In the case of FPS computation in the CPU, it is impossible to simulate insufficient performance. However, GPU-based implementation can handle the same situation more efficiently than CPU. In GPU computations for 10, 15, and 20 instances of each 3D model, minimal differences were observed due to the parallel processing capabilities of the GPU. When the number of objects increased to 180, there was a significant disparity in the results. Furthermore, the number of each model could not reach a larger number than 180 because the number of threads in the GPU count was over the allowed limit as shown in Figure 5.

Table 1. FPS comparison computation between CPU and GPU.

3D Model	Computation Comparison			
	Number of Objects	Average FPS in CPU	Average FPS in GPU	Times
Sphere 162 vertices 320 triangles	10	24.16	650.98	26
	15	12.34	645.25	52
	20	7.42	643.30	86
	180	–	532.71	–
Torus 2304 vertices 4608 triangles	10	6.83	638.34	93
	15	4.16	637.48	153
	20	2.9	635.62	219
	180	–	372.88	–
Bunny 2990 vertices 5976 triangles	10	5.51	635.25	115
	15	3.39	633.52	186
	20	2.39	629.19	263
	180	–	360.64	–
Armadillo 6362 vertices 12,720 triangles	10	2.91	623.26	214
	15	1.87	622.19	322
	20	–	619.23	–
	180	–	196.33	–

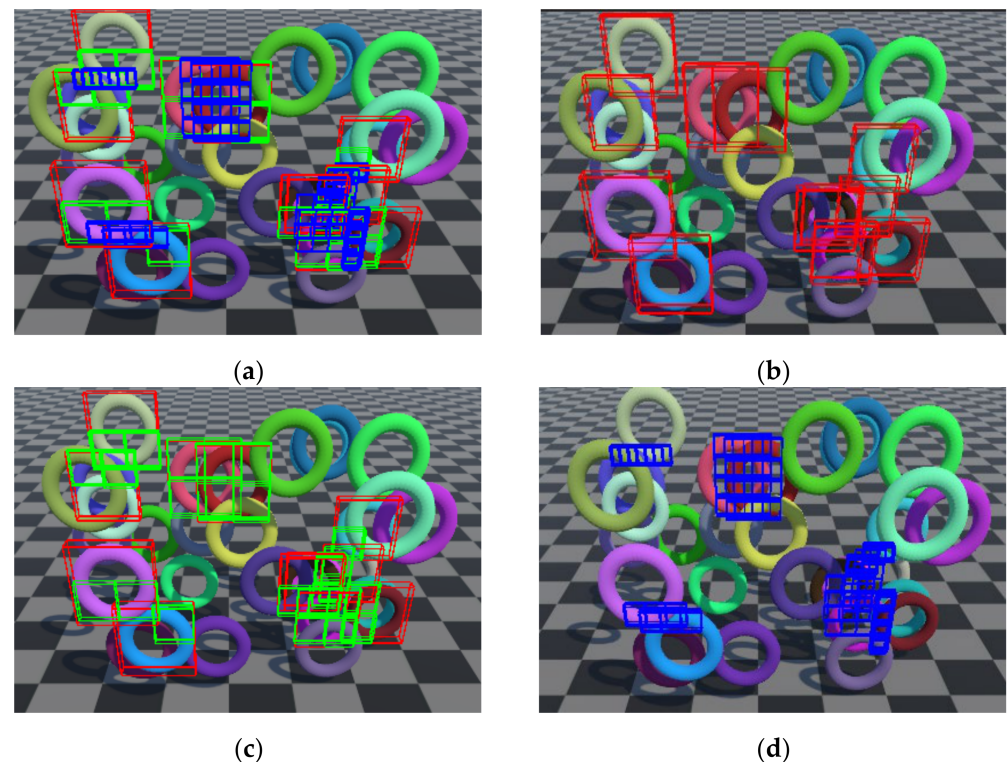


Figure 5. Experimental results from the Torus objects: (a) result of collision detection at all levels in Octree structure, (b) result of collision detection level zero, (c) result of collision detection level one, and (d) result of collision detection level two.

5. Conclusions

We developed the collision detection algorithm using the Octree-based AABB and compared the computation ability in CPU and GPU. The AABB algorithm calculated the bounding box for collision detection on objects by traversing all vertices and testing the intersection between the bounding boxes. The algorithm was implemented with an Octree-based structure for the all-pairs collision detection, the comparison of the two AABBs, and checking collision detection bounding boxes. Additionally, for parallel processing, a GPU compute shader was used. The computation speed of GPU-based was 26 times faster than CPU. In the FPS computation in CPU, the minimum cost of FPS was 1.87, and it was impossible to simulate insufficient performance. However, GPU-based implementation improved its performance in the same situation.

Author Contributions: M.H. provided conceptualization, project administration, and edited and reviewed the manuscript. T.K. analyzed and designed this study. K.H. implemented the simulation and wrote the original draft. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education under Grant NRF-2022R111A3069371, was funded by the BK21 FOUR (Fostering Outstanding Universities for Research) No. 5199990914048.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: No new data were created or analyzed in this study.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Nie, Q.; Zhao, Y.; Xu, L.; Li, B. A survey of continuous collision detection. In Proceedings of the IEEE International Conference on Information Technology and Computer Application (ITCA), Guangzhou, China, 18–20 December 2020; pp. 252–257.
2. Ströter, D.; Mueller-Roemer, J.S.; Stork, A.; Fellner, D.W. OLBVH: Octree linear bounding volume hierarchy for volumetric meshes. *Vis. Comput.* **2020**, *36*, 2327–2340. [[CrossRef](#)]
3. Wang, M.; Chen, S.; Yang, Q. Design and application of bounding volume hierarchy collision detection algorithm based on virtual sphere. *J. Mech. Med. Biol.* **2019**, *19*, 1940044. [[CrossRef](#)]
4. Melero, F.J.; Aguilera, A.; Feito, F.R. Fast collision detection between high resolution polygonal models. *Comput. Graph.* **2019**, *83*, 97–106. [[CrossRef](#)]
5. Sung, M. Visibility-Based Fast Collision Detection of a large number of Moving Objects on GPU. *IEEE Access* **2023**, *11*, 49456–49463. [[CrossRef](#)]
6. Aissa, N.; Douteau, L.; Abisset-Chavanne, E.; Digonnet, H.; Laure, P.; Silva, L. Octree Optimized Micrometric Fibrous Microstructure Generation for Domain Reconstruction and Flow Simulation. *Entropy* **2021**, *23*, 1156. [[CrossRef](#)] [[PubMed](#)]
7. Serpa, Y.R.; Rodrigues, M.A.F. Broadmark: A Testing Framework for Broad-Phase Collision Detection Algorithms. *Comput. Graph. Forum* **2020**, *39*, 436–449. [[CrossRef](#)]
8. Mihai, C.C.; Lupu, C. Using graphics processing units and compute shader in real time multimodel adaptive robust control. *Electronic* **2021**, *10*, 2462. [[CrossRef](#)]
9. Yang, Y.; Chen, X.; Han, Y. Dadu-CD: Fast and efficient processing-in-memory accelerator for collision detection. In Proceedings of the ACM/IEEE Design Automation Conference (DAC), San Francisco, CA, USA, 20–24 July 2020; pp. 1–6.
10. Va, H.; Choi, M.H.; Hong, M. Real-time cloth simulation using compute shader in Unity3D for AR/VR contents. *Appl. Sci.* **2021**, *11*, 8255. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.